A Flow-Aware Extension of KD-BERT for IoT Network Intrusion Detection: Design and Methodology

[Author One], [Author Two], and [Author Three]

Abstract—Transformer-based network intrusion detectors can represent byte-level packet structure, but a compact packet classifier may not explicitly preserve the short-term relationship among packet sources, targets, and arrival times. This paper specifies a flow-aware extension to the published KD-BERT method, with particular attention to distinguishing denial-of-service (DoS) and distributed denial-of-service (DDoS) classes when those labels are present. The reference implementation retains the central KD-BERT student and teacher-distillation path. The proposed student adds a packet-wise encoder, a gated recurrent unit over a five-packet window, coarse packet metadata, and a one-second destination/service context derived from stable pseudonyms scoped to each capture. An auxiliary binary head is trained on DoS and DDoS examples alongside the primary multiclass task. This manuscript documents the motivation, architecture, feature definitions, privacy treatment, implementation corrections, and a reproducible evaluation protocol. The supplied corpus is organized by label and contains one capture per label in the available project copy, so the proposed evaluation protocol uses chronological partitions within captures. This manuscript is a theory-oriented design and implementation specification; it contains no empirical model measurements.

Index Terms—Internet of Things, intrusion detection, denial-of-service, distributed denial-of-service, knowledge distillation, network traffic, Transformer, flow context.

I. Introduction

Internet of Things (IoT) networks place computing and sensing devices in environments where security monitoring must coexist with limits on memory, energy, and processing capacity. A network intrusion detector (NID) must also represent more than individual packet contents. Some attacks are expressed through byte patterns or protocol misuse; others are more visible through the timing, volume, and composition of a sequence of packets. These requirements motivate models that can combine packet-level representation with a small amount of temporal context.

Knowledge distillation offers one path from a high-capacity representation model to a smaller task model. The original KD-BERT paper applies this idea to IoT network intrusion detection. It represents packet header and payload bytes as tokens, pretrains a large Transformer teacher with masked packet modeling (MPM) and same-origin packet prediction (SPP), and transfers teacher knowledge to a compact student [1]. The approach provides a useful reference because it connects packet representation and model compression in a unified pipeline. Its core design, however, treats traffic primarily as packet-byte sequences arranged in a short window. The student input does not directly expose the number or diversity of sources contacting a particular destination during a recent interval.

That distinction is relevant to denial-of-service traffic. A DoS event may originate from a single source or a small set of sources, while a DDoS event is commonly associated with a larger or more diverse set of senders. In practice the patterns overlap: source addresses can be spoofed, hidden behind translation, or repeated across devices; distributed traffic can also be low-rate or synchronized. Therefore, source count alone is not a reliable classifier. It is one signal that can be interpreted with packet rate, byte rate, timing, protocol, destination service, and TCP control flags. The design in this paper adds those signals as compact context while retaining the original packet-byte learning path.

The main methodological change is to represent the same five packets in two complementary ways. The baseline concatenates their packet encodings into a single Transformer input. The proposed student encodes each packet separately, adds a small vector of packet metadata, aggregates packet vectors in time with a gated recurrent unit (GRU), and fuses that sequence representation with victim- and service-scoped statistics from the preceding second. A secondary head supplies a direct learning signal for the DoS-versus-DDoS distinction, while the primary head continues to predict the dataset's full label set.

The paper makes four contributions. First, it describes a corrected, executable KD-BERT baseline that preserves the published objectives while making segment handling, padding masks, negative construction, and validation-based checkpoint selection explicit. Second, it specifies a flow-aware student whose added inputs are derived from privacy-scoped packet records rather than raw endpoint strings. Third, it states the expected mechanism for each change and its limitations, especially where address spoofing or capture-specific traffic may weaken the signal. Fourth, it defines a reproducible protocol for a later empirical study without presenting performance claims. The intent is to make the design reviewable before model training and to separate proposed mechanisms from evidence that must still be collected.

[Department, Institution, City, Country. Add corresponding-author email and funding information before submission.]

II. Background and Related Work

A. Transformer Representations and Distillation

BERT introduced bidirectional Transformer pretraining with a masked language modeling objective, in which selected input tokens are hidden or replaced and the model reconstructs them from surrounding context [2]. KD-BERT adapts this style of learning to packet bytes and introduces traffic-oriented pretraining objectives. DistilBERT demonstrates a teacher–student recipe that combines task-level supervision with intermediate representation constraints to train a smaller model [3]. The present design follows the same broad compression motivation but keeps the original KD-BERT objectives in the reference path and treats the flow-aware student as a structurally different learner.

Network traffic has also been represented directly with Transformer encoders. ET-BERT pretrains contextual datagram representations from unlabeled traffic and then adapts them to encrypted-traffic tasks [4]. This line of work supports the premise that network packets can be modeled as structured token sequences. The proposed extension addresses a different modeling axis: instead of changing the basic idea of packet tokenization, it adds short-window context that describes relations among packets and sources. The purpose is to let the classification layer see both byte-derived content and selected traffic statistics.

B. Lightweight Detection and Temporal Features

Kitsune studies lightweight online network intrusion detection through an ensemble of small autoencoders and incremental traffic features [5]. Its design illustrates that traffic summaries can be maintained with bounded context and modest per-instance processing. The proposed model is supervised and Transformer-based rather than an online anomaly detector, but it shares the practical goal of keeping additional traffic context compact.

The GRU is a gated recurrent architecture for mapping sequences into state representations and was introduced as part of the recurrent encoder–decoder family by Cho et al. [6]. Here it aggregates five packet vectors, not natural-language tokens, and provides an ordered summary before the classification heads. For DDoS analysis, David and Thomas describe flow-based detection using entropy and an adaptive threshold [7]. Their method motivates treating source distribution and traffic variability as useful characteristics; the present design uses normalized source entropy as one learned input alongside other features. Entropy is not treated as a standalone decision rule because identical entropy values can arise from distinct traffic behaviors.

C. Positioning of the Proposed Method

The proposed method is not intended to replace KD-BERT's packet representation or teacher. It adds an explicit traffic-context pathway to the student. The distinction is important: the byte model can learn local content and header structure, while compact context variables make selected temporal properties directly observable to the classifier. The architecture retains a closed set of supervised labels. It does not perform open-set detection, online adaptation, response mitigation, or automatic discovery of an attack family that is absent from training labels.

III. Problem Scope and Design Objectives

Let a capture contain timestamped packets $p_1, \ldots, p_T$ and a class label inherited from the dataset organization. The primary prediction task assigns each window of five consecutive packets a label from the known set $\mathcal{Y}$. When the label set includes DoS and DDoS categories, an auxiliary task maps only those examples to a binary target. All other examples are excluded from that auxiliary loss.

The design is guided by five objectives:

1) Retain a method-aligned reference. The baseline must keep the original header–payload tokenization and teacher objectives so that the added flow pathway is clearly attributable to the extension.
2) Represent short-range ordering. Packet order and inter-arrival time should be available to the student without expanding every window into a single, undifferentiated sequence.
3) Expose source distribution. A compact representation should summarize how many distinct pseudonymous sources contact the window-ending destination and how traffic is distributed among them.
4) Limit identifying data. Raw IP addresses and raw transport-port values should not be serialized into the shared cache. Any endpoint linkability needed by the feature path should be scoped to a single capture.
5) Keep claims testable. Each architectural addition should have a corresponding ablation or diagnostic, and an apparent improvement on a chronological split should not be described as capture-independent generalization.

These objectives define a design proposal rather than evidence of efficacy. In particular, a larger source count could reflect legitimate load balancing, while a single attacker can create many apparent sources through spoofing. The intended benefit is therefore not that one feature proves that traffic is distributed, but that the model receives a richer and more explicit view of traffic composition.

IV. Reference Method: KD-BERT Baseline

A. Packet Representation

The baseline starts with the packet's network and transport representation. Ethernet framing is removed, the header is retained, and up to 80 bytes of payload are included in the tokenized view. Adjacent bytes are combined into 16-bit values and shifted above a reserved range for special symbols. The input uses classification, separator, and mask tokens, and it assigns segment identifiers to distinguish header bytes from payload bytes. This

keeps the original method's key representation idea: the Transformer receives a sequence that marks the transition from protocol structure to payload content.

A window contains $K = 5$ packets. The five packet encodings are concatenated, truncated or padded to a maximum sequence length of 256 tokens, and passed to the student. Padding positions are masked in attention and in losses that compare teacher and student activations. The token vocabulary has 65,540 entries, including the byte-pair values and special symbols.

B. Teacher Pretraining

The teacher configuration follows the provided KD-BERT implementation: twelve Transformer layers, hidden dimension 768, twelve attention heads, and feed-forward dimension 3072. It is trained on the training partition with two self-supervised tasks. MPM masks 15% of eligible tokens and predicts their original token identities. Under the usual masked-language replacement mixture, most selected tokens are replaced with a mask symbol, a smaller subset is replaced with another token, and the remainder is left unchanged while still being predicted. This objective encourages the model to use surrounding packet bytes and context.

SPP predicts whether a header–payload pairing is drawn from the same packet or has been mismatched. The negative construction must always choose a different packet; otherwise a supposed negative can be identical to the positive and contaminate the objective. The training code preserves the association between the original header and payload for positive examples and records the masking and segment information required to reproduce the task.

C. Student and Distillation

The reference student has two Transformer layers, hidden dimension 128, two attention heads, and feed-forward dimension 512. Intermediate teacher and student states are compared at aligned depths, with learned projections used where dimensions differ. The loss includes attention-map and hidden-state terms:

$$\mathcal{L}_{\mathrm{KD}}^{\mathrm{base}} = \sum_{m \in \{1,2\}} \left(\mathcal{L}_{\mathrm{attn}}^{(m)} + \mathcal{L}_{\mathrm{hid}}^{(m)} \right). \tag{1}$$

The classifier uses the final [CLS] state and a dropout layer. During supervised learning, the student is trained on the multiclass label of each five-packet window.

Several implementation safeguards are necessary for a meaningful reference implementation. First, segment identifiers must be passed to the embedding stack rather than merely created by the tokenizer. Second, packet sanitization must not remove the transport header along with the payload, because doing so would erase ports and flags before the designed fields are extracted. Third, padding positions must be excluded from teacher–student losses. Fourth, an SPP negative must come from a different packet. Finally, the selected model checkpoint should be determined by validation data and then held fixed for the test partition. These corrections improve methodological consistency and traceability; they are not presented as new learning objectives.

D. Scope of Alignment with the Source Article

The baseline preserves the central KD-BERT design choices that are available in the supplied implementation: a 12-layer teacher, a two-layer 128-dimensional student, header–payload byte-pair tokenization, five-packet windows, MPM and SPP pretraining, layer-mapped attention and hidden-state distillation, and the published training schedule. It should be described as an implementation-aligned reference, not as an exact reproduction of every experiment in the source article. The article reports experiments across TON-IoT, Edge-IIoTset, and IoMT2024, including continued pretraining with IoMT2024 traffic. The supplied notebooks instead initialize a local teacher for the current project corpus and do not load the authors' published checkpoint or reproduce their multi-dataset evaluation.

The split procedure also differs. The source article describes selecting a bounded number of windows per class and partitioning the resulting data into training, validation, and test subsets. The supplied notebooks assign packets to capture-level splits when enough captures exist; with the present single capture per class, they use chronological boundaries before constructing windows. This choice is intended to reduce leakage from overlapping neighboring windows across partitions. It still tests later portions of the same capture and is not evidence of generalization to new captures. These differences are part of the experimental specification and must remain visible in any later interpretation.

V. Proposed Flow-Aware Extension

A. Overview

The flow-aware student receives the same packet windows and teacher-generated targets as the baseline. It first tokenizes each packet separately. A small packet encoder produces one vector per packet, while a parallel metadata encoder represents length, timing, protocol, endpoint changes, TCP flags, and a coarse destination-service bucket. A GRU summarizes the five packet vectors in order. A second branch embeds a window-context vector computed from preceding packets that share the destination pseudonym, IP protocol, and service bucket of the last packet in the window. A fusion layer joins the GRU state and context embedding before the classification heads. Fig. 1 summarizes the two student paths and the shared data and teacher components.

B. Per-Packet Metadata

For each packet p_i in a window, the feature vector contains ten values: log-normalized packet length; log-normalized inter-arrival time; normalized IP protocol number; source-pseudonym change indicator; destination-pseudonym change indicator; TCP SYN, ACK, RST, and

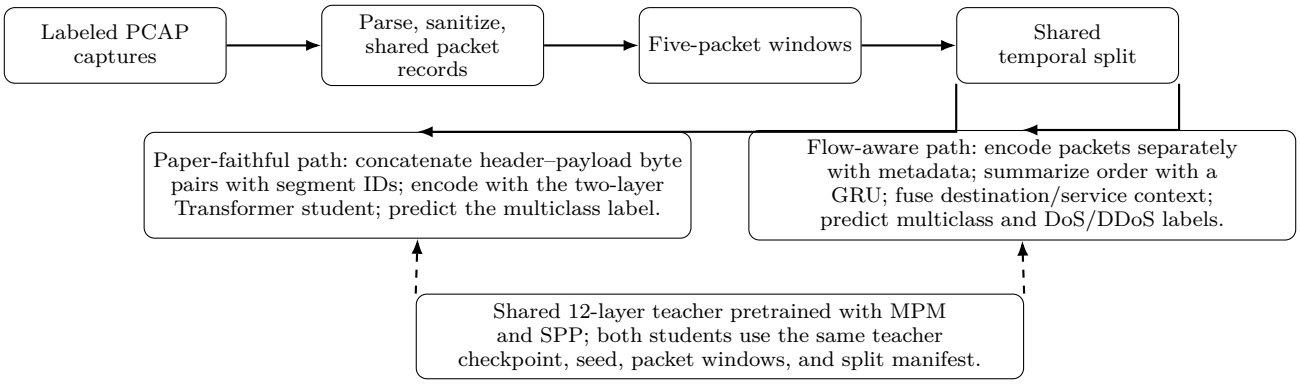


Fig. 1. Method overview. The baseline remains the reference pathway. The proposed pathway adds per-packet metadata, ordered aggregation, and target-scoped context while sharing the pretrained teacher and training partition.

FIN indicators; and the coarse destination-port bucket. Ports are grouped into service ranges rather than retained as raw integers. The available implementation maps the port to one of four coarse buckets: no port, well-known, registered, or dynamic/private range. This produces a small fixed-width vector even when port numbers vary widely.

Let $\Delta t_i = \max(0, t_i - t_{i-1})$ for adjacent packets in the window, with $\Delta t_1 = 0$. Packet size and inter-arrival time are transformed using bounded logarithmic scalings so that large values do not dominate the input. The protocol number and service bucket are linearly normalized. Change indicators are computed from capture-scoped pseudonyms and are zero for the first packet because no preceding packet exists inside the window. These features describe immediate transitions rather than global host identity.

C. Destination and Service Context

The context branch is scoped by the tuple (d^*, q^*, b^*) associated with the final packet in a window, where d^* is the pseudonymous destination, q^* is the network-layer protocol, and b^* is the coarse destination-service bucket. For a window ending at time t, the context includes matching packets in $[t-1, t]$ seconds and excludes any packet ordered after the window-ending packet. This constraint prevents the feature builder from using future traffic. It also means that the method summarizes a local temporal neighborhood rather than an entire capture.

Let $\mathcal{C}_t$ be the matching packet set in the one-second lookback. For each packet $j \in \mathcal{C}_t$, let s_j be its stable source pseudonym and ℓ_j its captured length. The context vector includes the number of unique sources, normalized source entropy, packet rate, byte rate, SYN fraction, mean inter-arrival time, protocol, and service bucket. The source distribution is

$$p(s) = \frac{|\{j \in \mathcal{C}_t : s_j = s\}|}{|\mathcal{C}_t|}, \qquad H_s = -\sum_s p(s) \log p(s), \tag{2}$$

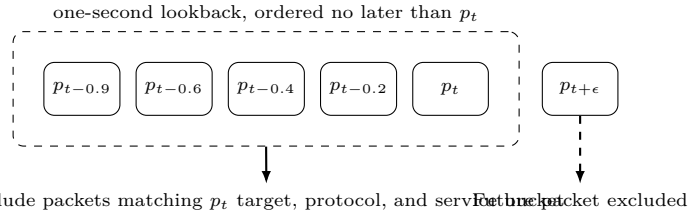


Fig. 2. Causal context construction for the window-ending packet. The context is destination/service scoped, bounded to one second, and excludes future packets.

with normalized entropy $\widetilde{H}_s = H_s / \log U$ for $U > 1$ and zero when there is at most one unique source. The traffic rates use the fixed one-second interval:

$$r_p = \frac{|\mathcal{C}_t|}{1\,\mathrm{s}}, \qquad r_b = \frac{\sum_{j \in \mathcal{C}_t} \ell_j}{1\,\mathrm{s}}. \tag{3}$$

The implementation applies logarithmic normalization to these counts and rates. A SYN fraction is the number of packets with the SYN bit divided by the number of packets in the context set. Mean inter-arrival time is zero when fewer than two timestamped packets are available.

Fig. 2 illustrates the temporal and destination/service scope. This context is motivated by a mechanism, not by an assumption that entropy uniquely identifies DDoS. A rise in unique sources or source entropy can accompany distributed activity, while high rate and SYN fraction may characterize a flood pattern. A single-source DoS can exhibit high packet rate but low source diversity. Spoofed addresses, NAT, benign flash crowds, or many-to-one service traffic can produce counterexamples. The learned classifier therefore receives the context jointly with the byte and sequence representations and can use combinations of cues.

D. Packet Encoder and Temporal Aggregation

Each packet is tokenized separately and padded to at most 64 tokens. The token embedding maps the 65,540-symbol vocabulary to dimension 32; a learned projection maps this representation to the student hidden size 128. Positional and segment embeddings are added, followed by

layer normalization and one Transformer block. A mask-aware mean pool converts each packet's token sequence into a fixed-length vector. The pool excludes padding positions, so the packet representation does not depend on the number of pad tokens.

The ten-dimensional packet metadata vector is transformed by a small multilayer perceptron and added to its packet vector. The sequence of five augmented packet vectors is then processed by a one-layer GRU. If h_i is the vector for packet i, its recurrent state follows the standard gated update form

$$z_i = \sigma(W_z h_i + U_z g_{i-1}), \quad r_i = \sigma(W_r h_i + U_r g_{i-1}), \quad (4)$$

$$\widetilde{g}_i = \tanh(W_h h_i + U_h(r_i \odot g_{i-1})), \quad g_i = (1 - z_i) \odot g_{i-1} + z_i \odot \widetilde{g}_i, \quad (5)$$

where g_i is the hidden state, z_i and r_i are update and reset gates, and σ is the logistic function. The final state g_K summarizes the ordered packet window. The equations describe the usual GRU operation; the implementation delegates this recurrence to the deep-learning library.

The eight-dimensional context vector is processed independently by a context MLP. The temporal state and context embedding are concatenated, transformed by a fusion MLP, and passed to a multiclass head. A second linear head predicts the DoS/DDoS subtype for eligible samples. The full flow representation can be written as

$$z = \mathrm{MLP}_f\left[g_K \,\|\, \phi(c_t)\right], \quad (6)$$

where c_t is the destination/service context and $\|$ denotes concatenation. The main classifier maps z to C class logits; the auxiliary classifier maps it to two subtype logits.

E. Distillation and Supervised Objectives

The flow-aware student has a different internal structure from the baseline, so its representation is not directly aligned with every Transformer layer of the teacher. A learned linear map W projects the fused flow representation to the teacher's hidden dimension. For teacher [CLS] representation h_T, the distillation term is

$$\mathcal{L}_{\mathrm{rep}} = \|Wz - h_T\|_2^2. \quad (7)$$

This objective transfers a task-window representation rather than forcing the student to reproduce attention maps that have no one-to-one correspondence with its GRU and context pathway.

For labeled training windows, the primary loss is multiclass cross entropy. For eligible DoS and DDoS windows, the subtype loss is binary cross entropy (implemented as a two-class cross-entropy head). The joint supervised objective is

$$\mathcal{L}_{\mathrm{sup}} = \mathcal{L}_{\mathrm{multi}} + \lambda_{\mathrm{aux}} \mathcal{L}_{\mathrm{subtype}}, \qquad \lambda_{\mathrm{aux}} = 0.2. \quad (8)$$

The auxiliary term is computed only where the label begins with a DoS-family label; samples from all other classes do not contribute to it. The combined training schedule first transfers teacher representations and then fine-tunes the student with labeled windows. This design is expected to preserve general packet discrimination while emphasizing a targeted distinction, but whether that balance is achieved requires empirical validation.

VI. Detailed Change Analysis Relative to KD-BERT

This section explains what the proposed implementation changes, why each change is introduced, and the mechanism by which it could help. The explanations are design rationales. They are not statements that the proposed model has already improved detection. The central principle is to preserve KD-BERT's packet representation and teacher knowledge while adding an explicit path for information that the reference student does not receive as a compact, named feature vector.

A. What Is Preserved

The extension retains the original paper's core learning structure: packet headers and payloads are represented as byte-pair tokens; the teacher learns from masked packet modeling and same-origin packet prediction; a smaller student learns from the teacher; and supervised prediction uses a short packet window. These choices are preserved so that the new method remains recognizably connected to KD-BERT and continues to model packet content end to end.

Keeping this foundation also establishes a useful conceptual control. If the packet tokenizer, teacher objectives, student depth, and window definition were all replaced at once, it would be difficult to explain the role of the flow features. The proposed design instead adds a parallel context pathway and changes the student organization where needed to consume that pathway. The teacher remains a source of transferable packet-level representations, while the flow-aware branch contributes temporal and source-distribution information.

The implementation should nevertheless be described as aligned with the central KD-BERT method, not as a complete reproduction of the source article. The source paper evaluates multiple datasets and uses continued pretraining; the supplied notebooks train a local teacher for the project corpus and use a split adapted to the supplied capture layout. They do not load the authors' published teacher checkpoint or reproduce the full multi-dataset experimental study. This distinction prevents an implementation comparison from being mistaken for a direct replication of the original paper.

B. From Per-Packet Address Randomization to Capture-Scoped Pseudonyms

The KD-BERT preprocessing strategy replaces original IP addresses and zeroes transport ports to limit identifying information and reduce address-specific effects. In the baseline view of the supplied implementation, the replacement address changes by packet, which prevents the classifier from directly linking repeated packets to the same source. This treatment is useful for privacy and discourages memorizing address strings, but it also

removes a relational signal: the model cannot count how many distinct sources contacted a destination if each packet appears to come from a new pseudonym.

The flow-aware view therefore computes a deterministic pseudonym from the experiment seed, capture identifier, and original endpoint address. The pseudonym remains stable inside that capture but is different across captures or seeds. The raw address is not written into the shared cache. This allows source-change indicators, unique-source counts, and source-frequency distributions to be computed without putting address text into the model input. It is a targeted compromise between complete packet-level unlinkability and retaining the short-lived relations needed for traffic analysis.

The scope is intentionally local. A capture-scoped pseudonym is not an anonymous identity: it reveals that several packets within one capture share an endpoint. The scope limits cross-capture linkage, but the hash is deterministic and may be vulnerable if its inputs are known or can be guessed. For deployments that require stronger privacy, the pseudonymization key should be secret and managed separately, or source-derived statistics should be computed inside a trusted preprocessing boundary and the identifiers discarded afterward. The design's expected benefit is that source distribution becomes observable; its cost is that within-capture linkability is introduced.

Port handling follows a related principle. The packet byte view has raw port values removed, but the flow branch keeps a coarse service category computed before removal. This lets the context distinguish broad destination-service groups without exposing the exact port number. The category may help separate traffic directed at different service types, while the coarse mapping reduces the chance that a model simply memorizes a dataset-specific port. It can still leak service information and should be included in the privacy review.

C. From a Concatenated Window to Packet-Wise Encoding and Ordered Aggregation

In the reference method, packet encodings from a window are concatenated and processed as one long Transformer input. This gives self-attention access to tokens from all packets, but it mixes two different relationships in one sequence: relationships among bytes within a packet and relationships among packets over time. The proposed student first encodes each packet independently, pools its valid token states, adds packet metadata, and then processes the ordered packet vectors with a GRU.

This separation gives the architecture an explicit hierarchy. The packet encoder learns local structure such as header patterns and bounded payload context. The recurrent layer learns how packet-level states change across the window. The first stage is therefore responsible for within-packet representation, while the second stage is responsible for packet order. The intended advantage is a clearer inductive bias for traffic sequences and a smaller operation over packet vectors rather than a single self-attention operation over every token in the entire window.

The GRU is chosen because the window is short and order matters. Its update and reset gates can retain or replace information as successive packets arrive. The design does not assume that recurrence is universally superior to attention; it expresses the hypothesis that an explicitly ordered, compact state may be a suitable way to aggregate a small number of packet representations. A later study can remove the GRU or replace it with masked pooling to determine whether ordered recurrence is useful for the task.

D. Why Add Packet Metadata

Byte tokens can encode packet length, flags, and protocol values implicitly, but the classifier must discover their meaning from the byte representation. The proposed model supplies a small vector of normalized metadata alongside the packet embedding. The vector includes packet length, inter-arrival time, network protocol, source and destination change indicators, selected TCP flags, and a coarse destination-service bucket.

This change makes several traffic properties directly accessible to the downstream sequence encoder. Packet length can describe the size profile of a burst. Inter-arrival time exposes whether packets are concentrated in time. Protocol and service categories distinguish broad transport contexts. Source and destination change indicators describe local transitions in the window, and TCP control flags can expose patterns such as repeated connection initiation. These features are not substitutes for packet bytes; they are auxiliary descriptions intended to reduce the burden of inferring simple statistics from a short token sequence.

The features are normalized with bounded logarithmic transforms where appropriate. A logarithm compresses a wide count or size range, and clipping limits the influence of extreme values. This is a stability measure, not a guarantee that the features are comparable across different capture environments. The chosen normalization constants and feature definitions must remain fixed for the paired notebooks, and any deployment on a different network may require recalibration using training data only.

E. Why Add a Destination- and Service-Scoped Lookback

The packet-wise metadata describes only the packets inside the five-packet window. A denial-of-service pattern can depend on activity that extends beyond that small window, especially when many sources send to one destination over a brief interval. The context branch therefore summarizes earlier packets from a one-second lookback that share the destination pseudonym, protocol, and coarse service bucket of the final packet. The window endpoint is the reference time, and packets later than that endpoint are excluded.

The context vector includes unique-source count, normalized source entropy, packet rate, byte rate, SYN fraction, mean inter-arrival time, protocol, and service

bucket. The source count describes the number of observed pseudonyms. Entropy describes how evenly packets are distributed among those pseudonyms: a single dominant source yields a more concentrated distribution, while traffic divided among many sources can yield a more diffuse distribution. The rates describe how much packet and byte volume is directed to the scoped target. Timing and SYN fraction add temporal and transport-specific context.

These quantities are complementary rather than definitive. A high source count may result from spoofed addresses or a benign flash crowd. A high rate may characterize either a malicious flood or a legitimate event. Entropy is sensitive to the source-address representation and the number of packets observed in the lookback. By conditioning all summaries on destination, protocol, and service, the design seeks a more relevant local description than a capture-wide statistic, but that scope can miss campaigns that move across targets or services. The expected improvement is better access to victim-centric context, not a deterministic rule that labels all distributed traffic correctly.

The context construction also makes causality explicit. A model prediction for the window-ending packet must not depend on traffic that has not yet arrived. Enforcing the time cutoff avoids future leakage in the feature builder and makes the same definition usable for an online detector. It does not by itself prove that all other training operations are leakage-free; the split and window-generation order must also be audited.

F. Why Add an Auxiliary DoS/DDoS Head

The primary classifier predicts the complete set of directory labels. That objective can devote most of its learning capacity to separating broad classes, even when the research question concerns confusion between two related denial-of-service families. The proposed auxiliary head supplies an additional supervised signal only for samples labeled as DoS or DDoS. It shares the fused representation with the main classifier but has its own two-class output layer.

The reason for this change is to make the targeted distinction explicit in the training objective. The main head still learns the full class taxonomy, and the auxiliary head focuses on the subtype distinction. This may encourage the shared representation to preserve features that separate the two labels while retaining the general task. The auxiliary loss is masked for samples outside those classes so that unrelated classes are not assigned an artificial subtype label.

This formulation depends on trustworthy labels and adequate examples of both classes. If a capture contains mixed attack behavior but receives a single directory label, the auxiliary objective can reinforce an incorrect boundary. The loss weight also creates a tradeoff: too little weight may provide no useful pressure, while too much may distort the main classification representation. The configured weight is an implementation setting to be treated as a design choice, not evidence of a preferred value.

G. Why Factorize the Token Embedding

The vocabulary table maps each byte-pair token to the student hidden dimension. Since the vocabulary is large, this lookup table can account for a meaningful part of the student parameters. The flow-aware branch first maps tokens to a narrower embedding and then projects the result into the model hidden dimension. Positional and segment information are added after projection.

The factorization separates storage width from computation width. A smaller lookup can reduce the embedding-table footprint while allowing later packet processing to use the wider hidden state. This is a direct architectural property. It does not guarantee lower end-to-end memory use or faster inference, because transformer operations, recurrent operations, intermediate activations, and implementation overhead also contribute to cost. The design is included as a resource-conscious hypothesis that should be measured on target hardware.

H. Why Distill a Pooled Representation in the Flow-Aware Branch

The KD-BERT baseline has Transformer layers that can be aligned with selected teacher layers. The flow-aware student contains a per-packet Transformer block, a recurrent sequence encoder, a metadata path, and a context path. There is no natural one-to-one correspondence between all of those internal states and the teacher's Transformer attention maps. Forcing layer-by-layer attention matching on this different topology could constrain the student to imitate structures it does not contain.

The proposed branch instead projects its fused window representation to the teacher hidden dimension and learns to match the teacher's final [CLS] representation for the same window. This gives the student a common target at the task-window level while leaving the internal flow pathway free to encode temporal context in its own form. The reference baseline retains attention and hidden-state distillation, because its layers are designed to align with teacher layers. The distinction in distillation objectives follows from the difference in student topology, not from a claim that one distillation loss is universally better.

I. Why Correct the Baseline Implementation

A fair method comparison requires the reference branch to implement the intended paper method consistently. The supplied implementation explicitly passes segment IDs to the embedding stack, preserves transport headers when separating payload bytes, masks padding positions in representation losses, constructs SPP negatives from a different packet, and selects a checkpoint using validation data. These changes do not add a new feature pathway.

They prevent implementation details from silently changing the baseline's input or training objective.

This distinction matters when interpreting a future comparison. If the baseline omitted segment identifiers or compared teacher and student states at padding positions, the flow-aware branch could appear to benefit from correcting code rather than from adding traffic context. The corrected reference therefore improves methodological clarity and makes the reason for any later model behavior easier to investigate. Checkpoint selection is performed on validation data, and the held-out test split should remain unused until the configuration is fixed.

J. Why Change the Split Procedure

The original article describes sampling a bounded number of sliding windows per class and partitioning the selected data. The supplied implementation creates capture-level partitions when multiple independent captures are available. If a class contains only one capture, it instead assigns packets chronologically to training, validation, and test partitions before creating windows. This order is intended to keep overlapping or adjacent windows from the same trace from being placed on opposite sides of a split.

The change addresses a common leakage path in packet-window experiments: windows with stride one share most of their packets with neighboring windows. Randomly dividing those windows can place near-duplicates into training and test data. Chronological partitioning reduces this particular overlap, while capture-level partitioning is stronger because it withholds a complete trace. The supplied corpus has one capture per label, so the fallback remains an evaluation of later traffic from traces that were partly observed during training. It does not establish generalization to unseen capture sessions. This limitation should be treated as a constraint on the scope of any future claim.

K. Meaning of Improvement in This Design

The proposed method is intended to be better specified for a targeted modeling question, not presumed to be better in measured performance. It makes source distribution, local timing, burst volume, and selected TCP behavior explicit inputs; it provides a dedicated learning signal for the DoS/DDoS distinction; and it organizes packet content and packet order in separate stages. These changes can make relevant cues accessible to the classifier and make the design easier to inspect.

Each change can also fail. Address spoofing can make source diversity misleading. A fixed lookback can miss slower behavior. Coarse service buckets can merge distinct applications. The auxiliary objective can compete with the main task. The GRU can add compute without useful information. The factorized embedding can reduce one component of model size while leaving total runtime unchanged. For these reasons, the paper describes the proposed mechanisms as hypotheses and specifies component-level diagnostics for future study. It does not equate a plausible rationale with demonstrated benefit.

VII. Dataset Handling and Experimental Protocol

A. Corpus Layout and Record Schema

The provided folder is organized into class directories labeled MITM, XSS1, backdoor, ddos, dos, injection, normal, password, runsomware, and scanning. Each directory in the supplied copy contains one PCAP capture. This corpus description is included to clarify the available evaluation scope, not as a model outcome. A record stores two sanitized header views, a bounded payload byte string, capture-relative path, packet index, timestamp, class label, stable pseudonymous endpoint identifiers, IP protocol, coarse port buckets, TCP flags, and captured packet length. Raw IP addresses are not stored in the cache. The exact provenance and label-generation procedure for the supplied captures should be added by the dataset owner before external publication.

The packet cache is shared by the two notebook variants. A content-based fingerprint is generated from each capture's relative path, file size, and SHA-256 digest. This makes the cache reusable when files are copied and their modification times change, while still detecting different capture contents. The split manifest is tied to the same fingerprint and random seed. If the cache is used without raw PCAP files, the notebook relies on the cache metadata that accompanies it; the experiment archive should preserve both files together.

B. Split Construction

The preferred split is capture-level assignment within each class, which reduces the chance that near-duplicate traffic from one capture appears in both training and test data. That method requires at least three independent capture files per class. The supplied corpus has only one capture for every class, so the implementation uses a chronological fallback: packets are sorted by timestamp and packet index, then assigned sequentially to training, validation, and test partitions in an 80/10/10 proportion. Windows are generated after packet assignment and never cross a capture or split boundary.

This fallback tests the later portion of each supplied trace after training and validation on earlier portions of the same trace. It does not measure generalization to a new device, capture session, network topology, or campaign. Temporal autocorrelation may also make adjacent windows similar. These constraints are stated as part of the protocol rather than hidden by a random packet-level shuffle. Additional independent captures are required for a stronger deployment-oriented study.

The window size is five packets and the stride is one packet. To control the training sample budget, at most 5,000 windows are retained per class across the three partitions, with the available budget apportioned using the configured 80/10/10 ratios. If a class has fewer windows than its cap, all available windows are retained. Sampling

is deterministic under seed 42. Both variants use the same selected window identifiers so that input changes are not confounded with a different sample set.

C. Matched Training Configuration

The teacher uses twelve layers, hidden dimension 768, twelve attention heads, and feed-forward dimension 3072. The baseline student has two layers, hidden dimension 128, two attention heads, and feed-forward dimension 512. The flow-aware student uses hidden dimension 128, a 32-dimensional token embedding, a single packet Transformer block, a one-layer GRU, and two classification heads. The teacher pretraining budget is 500,000 updates. The configured effective batch size is 32, implemented as microbatches of eight with gradient accumulation of four. The pretraining, distillation, and fine-tuning learning rates are each 2×10^{-5}; the supervised distillation and fine-tuning stages each use ten epochs. The random seed is 42. A smoke-run flag exists for debugging and must not be substituted for the full protocol in a research study.

The baseline notebook is run first and saves the shared teacher checkpoint and data split. The flow-aware notebook then reuses those artifacts. Both runs must use identical dataset fingerprint, split manifest, seed, window identifiers, and training schedule. The only intended change in the paired method is the student input organization and architecture. Runtime, package versions, accelerator type, and checkpoint provenance should be recorded in the experiment manifest to make subsequent repetitions interpretable.

D. Evaluation Plan

A later empirical study should use independent capture sessions and hold out complete captures from training. For the targeted question, the analysis should inspect errors between DoS and DDoS labels alongside false alarms on benign traffic, confidence behavior, and inference cost on a clearly specified device. Repeated seeds and a held-out capture source are needed before drawing claims about robustness or deployment usefulness.

Planned ablations remove one design element at a time: (i) source count and entropy; (ii) rate and timing features; (iii) TCP flags and port buckets; (iv) the auxiliary subtype head; and (v) the GRU. An additional variant can replace the context branch with a simple pooled packet window to isolate whether the destination/service lookback is useful. Each ablation should preserve the seed, selected windows, teacher checkpoint, and optimization schedule. These prospective ablations should preserve the seed, selected windows, teacher checkpoint, and optimization schedule. Removing source count and entropy asks whether source-distribution context contributes beyond rate and packet content. Removing rate and timing features isolates the value of short-interval traffic intensity. Removing TCP flags and service buckets examines coarse transport context. Removing the auxiliary head tests whether direct subtype supervision changes the shared representation. Removing the GRU tests whether ordered aggregation contributes beyond pooled packet vectors. Replacing the destination/service lookback with a pooled packet window examines whether the scoped context is useful. These checks define future experiments and are not claims about outcomes.

VIII. Privacy, Security, and Operational Considerations

The preprocessing pipeline creates two pseudonymous endpoint views. The randomized-pseudonym view varies the pseudonym by packet, which limits repeated-address linkage in the student byte representation. The flow-aware view uses a stable pseudonym within one capture so that source counts and endpoint-change indicators can be computed. Both views are scoped by capture and seed, and the raw address is not serialized in the cache. Conceptually, an endpoint identifier is derived as $u = \mathrm{Trunc}_{80}(\mathrm{SHA256}(s\|c\|a))$, where s is the experiment seed, c is the capture-relative identifier, and a is the original address. This transformation is pseudonymization, not a proof of anonymity: stable IDs reveal repeated communication within a capture and may be vulnerable to auxiliary information or deterministic re-identification if the keying inputs are known.

Payload bytes can contain application data, credentials, or personal information. The configured payload cap limits the number of retained bytes but does not guarantee that retained content is harmless. A deployment or publication pipeline should review payload retention, restrict access to the cache, and consider masking or omitting application payload where the threat model requires it. Dataset access and usage rights should be checked before redistribution.

The model itself is a classifier, not a mitigation engine. It does not block traffic, rate-limit sources, or distinguish legitimate flash crowds by consulting application state. A production system would need a policy layer, monitoring for drift, and a way to route uncertain or novel traffic to an analyst or independent detector. Since the training label set is closed, an unseen attack family may be assigned to the closest known category or to benign; the model should not be described as automatically adapting to previously unseen attacks.

IX. Expected Mechanisms and Limits

The proposed changes correspond to several testable mechanisms. First, source count and source entropy may help represent whether traffic to a victim is concentrated or spread across observed sources. Their utility is conditional on address integrity and capture scope. Second, packet and byte rates, mean inter-arrival time, and SYN fraction may represent short-lived load and transport behavior that a five-packet byte sequence may not expose consistently. Third, the GRU can preserve packet order while remaining smaller than a second large Transformer over the entire concatenated window. Fourth, the auxiliary subtype head can direct part of the supervised gradient toward the specific DoS/DDoS distinction without changing the primary multiclass output. Fifth, factorizing the

token embedding into a 32-dimensional lookup followed by a projection to 128 dimensions is intended to reduce the vocabulary table while keeping the downstream hidden width.

These mechanisms are hypotheses, not conclusions. The auxiliary head may harm other labels if it dominates shared features; the context vector may encode capture-specific artifacts; entropy may be noisy when few packets match the target key; and the GRU may add latency or provide little value on short windows. The proposed ablations are designed to locate those failure modes. If the attack family labels are inconsistent, no architecture can make the subtype objective meaningful. If the data contain only one capture per label, validation and test partitions may share temporal and environmental properties with training traffic.

X. Limitations and Threats to Validity

The supplied corpus has one capture per class. The chronological fallback is preferable to shuffling packets from the same trace across partitions, but it does not establish cross-capture generalization. Traffic may be ordered by attack phase, and future windows may differ systematically from earlier ones. A future study should collect or obtain multiple captures per label, preferably from different devices, sessions, network configurations, and attack realizations, then reserve entire captures for final evaluation.

The label taxonomy and source provenance require external confirmation. A directory name is treated as the ground-truth class for every IP packet in its capture. If a capture mixes benign and malicious traffic or contains multiple attack phases, this labeling rule can introduce noise. In particular, the proposed auxiliary task depends on consistent DoS and DDoS labels. The method does not distinguish the number of physical attacking hosts from the number of observed source addresses, and NAT or spoofing can weaken that relation.

The context scope uses a fixed one-second window and a fixed destination/protocol/service key. This provides a bounded and reproducible feature but may miss slower attacks, multi-service campaigns, or patterns that shift destinations. The five-packet window likewise favors local patterns and may not summarize longer sequences. The implementation includes a maximum sample budget per class, which controls training size but can discard windows from large captures; deterministic sampling supports reproducibility but does not remove sampling bias.

The baseline and proposed student differ in multiple linked choices: packet-wise encoding, factorized embeddings, GRU aggregation, context features, fusion, and an auxiliary head. A paired main comparison can assess the complete design, while the planned ablations are needed to attribute any later behavior to individual components. Without those ablations, the contribution of each feature group would remain uncertain. Runtime claims also require direct measurement on specified hardware; model dimensions alone do not establish energy, memory, or latency suitability for a deployed IoT gateway.

XI. Conclusion

This paper specifies a flow-aware extension to KD-BERT for IoT intrusion detection. It retains a method-aligned teacher–student baseline and adds a compact pathway for packet metadata, ordered packet aggregation, destination/service-scoped source statistics, and auxiliary DoS/DDoS supervision. The design is motivated by the possibility that temporal rate and source-distribution cues complement byte-level representations, particularly when a detector must distinguish concentrated from distributed traffic. The proposed mechanisms remain hypotheses. A reliable assessment requires complete training, independent captures, verified labels, repeated seeds, and component ablations.

Appendix A
Feature and Record Definitions

For reproducibility, the flow-aware record exposes the following fields. header_paper contains a packet header with packet-specific address pseudonyms; header_flow contains a header with stable capture-scoped endpoint pseudonyms; payload is the bounded payload byte string; capture_id is the relative capture path; index is the packet order in the capture; timestamp is the capture timestamp; label is the directory label; src_id and dst_id are stable pseudonymous endpoint identifiers; protocol is the network-layer protocol number; src_port_bucket and dst_port_bucket are coarse port categories; tcp_flags is the TCP flag bitmask; and packet_length is the captured packet length. Raw addresses and raw port values are not stored in the cache.

The packet metadata is a $K \times 10$ array. Its columns are normalized length, normalized inter-arrival time, normalized protocol, source-change flag, destination-change flag, SYN flag, ACK flag, RST flag, FIN flag, and normalized destination-service bucket. The context representation is an eight-dimensional vector: logarithmic unique-source count, normalized source entropy, logarithmic packet rate, logarithmic byte rate, SYN fraction, logarithmic mean inter-arrival time, normalized protocol, and normalized destination-service bucket. The feature code clamps logarithmically scaled values to a fixed interval so that the input range remains bounded.

Appendix B
Implementation Alignment Checklist

Before a full run, the implementation should confirm that both notebooks resolve to the same dataset fingerprint, split manifest, class ordering, random seed, teacher checkpoint, and training budget. The cache should be loaded rather than regenerated when its content fingerprint matches. The baseline must receive segment IDs and mask padding in its losses. The SPP negative must be selected from a distinct packet. The flow context must

use only packets no later than the window end and must be grouped by the target pseudonym, protocol, and coarse service bucket. The auxiliary loss must be masked to DoS/DDoS examples. Validation data should select checkpoints, and the test partition should be held aside until the model configuration is fixed.

References

[1] Z. Cao, X. Liu, Z. Zhou, L. Ding, and W. Shang, "KD-BERT: A lightweight knowledge distillation bidirectional encoder representations from transformers for IoT network intrusion detection," IEEE Transactions on Industrial Informatics, vol. 21, no. 11, pp. 8475–8483, Nov. 2025.

[2] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, 2019, pp. 4171–4186.

[3] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "DistilBERT, a distilled version of BERT: Smaller, faster, cheaper and lighter," in Proceedings of the 5th Workshop on Energy Efficient Machine Learning and Cognitive Computing at NeurIPS, 2019. [Online]. Available: https://arxiv.org/abs/1910.01108

[4] X. Lin, G. Xiong, G. Gou, Z. Li, J. Shi, and J. Yu, "ET-BERT: A contextualized datagram representation with pre-training transformers for encrypted traffic classification," in Proceedings of the ACM Web Conference 2022, 2022, pp. 633–642.

[5] Y. Mirsky, T. Doitshman, Y. Elovici, and A. Shabtai, "Kitsune: An ensemble of autoencoders for online network intrusion detection," in Proceedings of the Network and Distributed System Security Symposium, 2018.

[6] K. Cho, B. van Merrienboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, "Learning phrase representations using RNN encoder–decoder for statistical machine translation," in Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing, 2014, pp. 1724–1734.

[7] J. David and C. Thomas, "DDoS attack detection using fast entropy approach on flow-based network traffic," Procedia Computer Science, vol. 50, pp. 30–36, 2015.